Job Scheduling Optimization

Problem 1.2.16 — EC 2007

CH Taraka Abhinav - EE25BTECH11016

Problem Statement

Four jobs have to be sequenced on a single facility, with the objective of minimizing the maximum tardiness

$$(\max_i |CompletionTime_i - DueDate_i|)$$

The jobs have due dates and processing times as follows:

Job	Due Date (d_i)	Processing Time (p_i)
P	5	2
Q	6	10
R	3	3
S	7	4

The **last** job that should be taken up is

(A) P

(B) Q

(C) R

(D) S

Understanding the Objective

We need to minimize the maximum absolute deviation from due dates:

$$\min \left(\max_i |C_i - d_i| \right)$$

Key Definitions:

- **Completion time (C_i):** Time at which job i finishes processing.
- **Lateness (L_i):** $L_i = C_i - d_i$
- **Absolute Lateness:** $|L_i| = |C_i - d_i|$

Observations:

- If $C_i > d_i \Rightarrow$ job is Late
- If $C_i < d_i \Rightarrow$ job is Early
- The objective behaves exactly like minimizing the **Maximum Lateness** ($L_{\max}$).

Alternative Scheduling Methods

In operations research, several heuristic rules are taught for single-machine scheduling. Why don't we use them here?

- **SPT (Shortest Processing Time):** Sort by increasing p_i.
  - *Pros:* Minimizes average completion time (mean flow time).
  - *Cons:* Ignores due dates entirely. Bad for lateness.
- **LST (Least Slack Time):** Sort by increasing $(d_i - p_i)$.
  - *Pros:* Good for minimizing maximum tardiness in some scenarios.
  - *Cons:* Does not guarantee optimal $L_{\max}$.
- **EDD (Earliest Due Date):** Sort by increasing d_i.
  - *Pros:* Mathematically proven to minimize maximum lateness ($L_{\max}$).

Comparison: Why SPT Fails Here

Let's try the **SPT (Shortest Processing Time)** rule: Sequence by smallest p_i: **P(2)** $\rightarrow$ **R(3)** $\rightarrow$ **S(4)** $\rightarrow$ **Q(10)**

Job	C_i **(Cumulative)**	d_i	**Absolute** ($\lvert C_i - d_i \rvert$)
P	2	5	$\lvert 2-5 \rvert = 3$
R	$2+3 = 5$	3	$\lvert 5-3 \rvert = 2$
S	$5+4 = 9$	7	$\lvert 9-7 \rvert = 2$
Q	$9+10 = 19$	6	$\lvert 19-6 \rvert = \mathbf{13}$

Maximum Deviation using SPT = **13**

Because Q takes 10 units but is due at 6, pushing it to the end creates a massive delay. We need a rule that respects due dates!

The Optimal Rule: EDD

For minimizing maximum absolute deviation ($L_{\max}$), we must use Jackson's Theorem:

EDD Rule (Earliest Due Date First)

Algorithm Idea:

- **Sorting:** Jobs are sequenced in increasing order of their due dates (d_i).
- **Complexity:** Runs in $\mathcal{O}(n \log n)$ time.

Important Result:

$$\boxed{\text{EDD guarantees the minimum possible } L_{\max}}$$

Step 1: Apply EDD to Given Data

Let's look at the given jobs and their due dates:

Job	Due Date (d_i)	Processing Time (p_i)
P	5	2
Q	6	10
R	3	3
S	7	4

Sort by increasing due date (d_i):

$$R\,(d_R = 3), \quad P\,(d_P = 5), \quad Q\,(d_Q = 6), \quad S\,(d_S = 7)$$

$$\Rightarrow \text{Optimal Sequence: } R \to P \to Q \to S$$

Step 2: Compute Completion Times

Cumulative Calculation:
Since jobs are processed one after another without idle time, the completion time (C_i) is the sum of processing times.

$$C_R = p_R = \mathbf{3}$$
$$C_P = C_R + p_P = 3 + 2 = \mathbf{5}$$
$$C_Q = C_P + p_Q = 5 + 10 = \mathbf{15}$$
$$C_S = C_Q + p_S = 15 + 4 = \mathbf{19}$$

Note: Total time taken to finish all jobs (Makespan) is 19 units.

Step 3: Compute Lateness and Absolute Values

Using the formula: $L_i = C_i - d_i$

Job	C_i	d_i	**Lateness** (L_i)	**Absolute** ($\lvert C_i - d_i \rvert$)
R	3	3	0	**0P5500Q156 + 99S197 + 1212hei**

Evaluating the Maximum Deviation:

$$\max_i |C_i - d_i| = \max(0, 0, 9, 12) = \mathbf{12}$$

The maximum lateness in this optimal sequence is 12 (Better than SPT's 13).

Final Conclusion

The optimal sequence using Earliest Due Date (EDD) is:

$$R \rightarrow P \rightarrow Q \rightarrow S$$

The question asks for the **last job** executed in the optimal schedule.

$$\text{Last Job} = \boxed{\text{Job } S}$$

Final Answer:

$$\boxed{\textbf{(D) S}}$$

Python: Data Setup & Analysis

```
import matplotlib.pyplot as plt
import numpy as np

# Job Data
jobs = {
   'P': {'p': 2, 'd': 5},
   'Q': {'p': 10, 'd': 6},
   'R': {'p': 3, 'd': 3},
   'S': {'p': 4, 'd': 7}
}

# Sequences
seq_edd = ['R', 'P', 'Q', 'S']
seq_spt = ['P', 'R', 'S', 'Q']
seq_lst = ['Q', 'R', 'S', 'P']

def analyze_sequence(seq):
   current_time = 0
   gantt = []
   abs_lateness = {}
   actual_lateness = {}
   cumulative_times = [0]
```

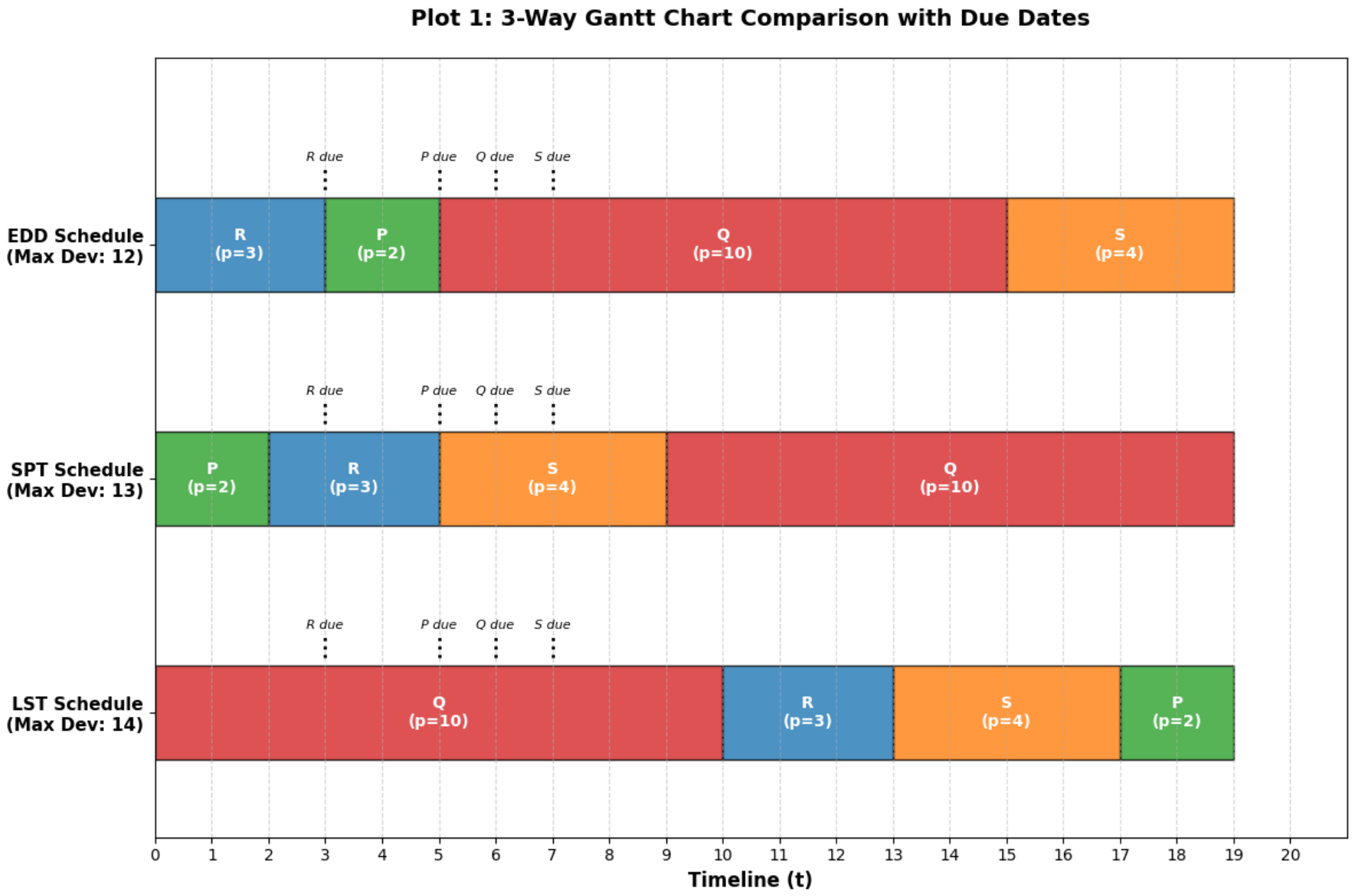
Python: Data Setup & Analysis

```
for job in seq:
    p = jobs[job]['p']
    d = jobs[job]['d']
    start = current_time
    current_time += p

    gantt.append((job, start, p, d))
    abs_lateness[job] = abs(current_time - d)
    actual_lateness[job] = current_time - d
    cumulative_times.append(current_time)

return gantt, abs_lateness, actual_lateness, cumulative_times
```

Python: Gantt Chart Generation

```
gantt_edd, _, _, _ = analyze_sequence(seq_edd)
gantt_spt, _, _, _ = analyze_sequence(seq_spt)
gantt_lst, _, _, _ = analyze_sequence(seq_lst)

colors = {'P': '#2ca02c', 'Q': '#d62728',
       'R': '#1f77b4', 'S': '#ff7f0e'}

fig1, ax1 = plt.subplots(figsize=(12, 8))

def draw_gantt_row(ax, gantt_data, y_base):
   for job, start, p, d in gantt_data:
      ax.broken_barh([(start, p)], (y_base, 6))
      ax.text(start + p/2, y_base + 3, job)

draw_gantt_row(ax1, gantt_lst, 5)
draw_gantt_row(ax1, gantt_spt, 20)
draw_gantt_row(ax1, gantt_edd, 35)

plt.savefig("G.png")
```

Gantt Chart Comparison

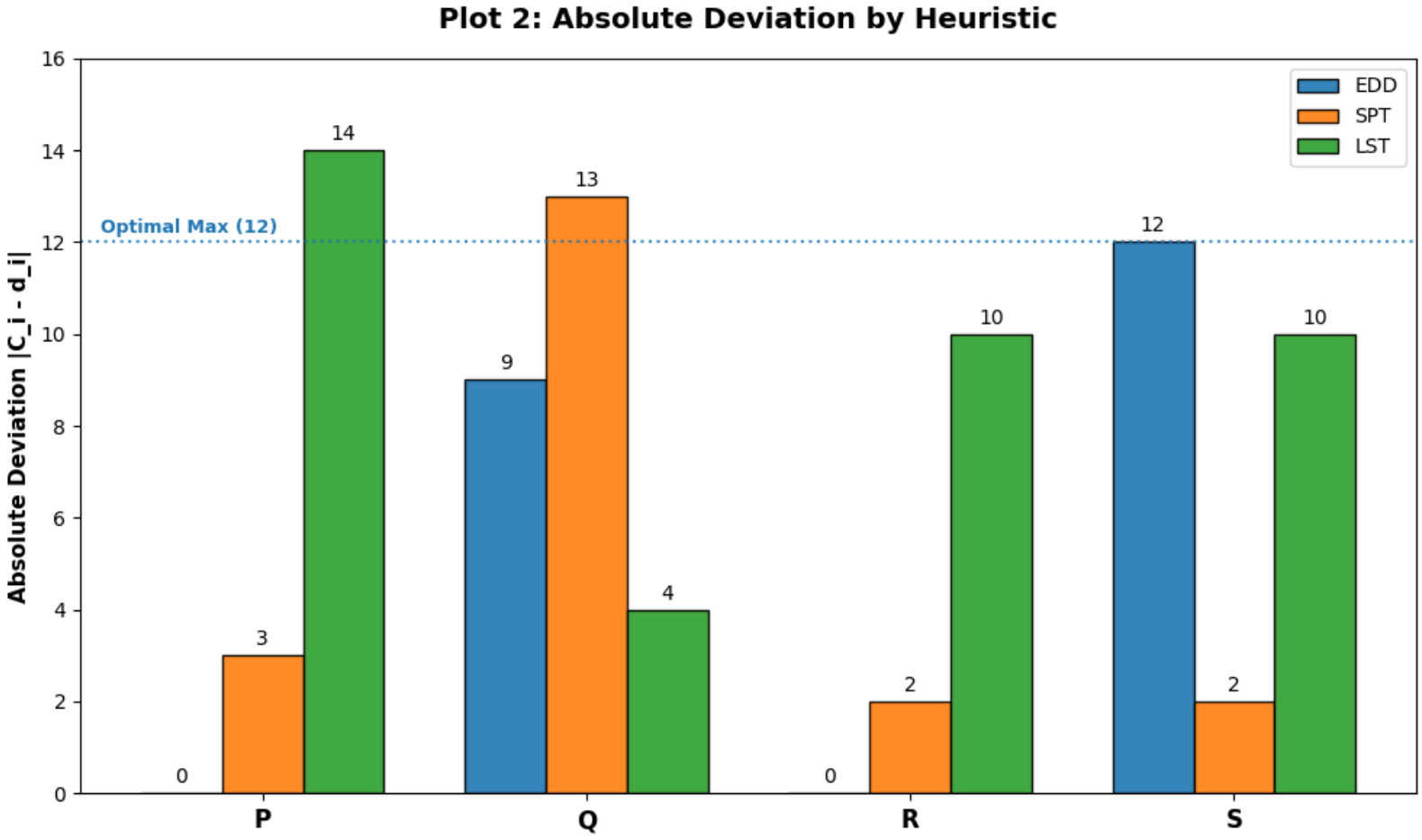


Observation:

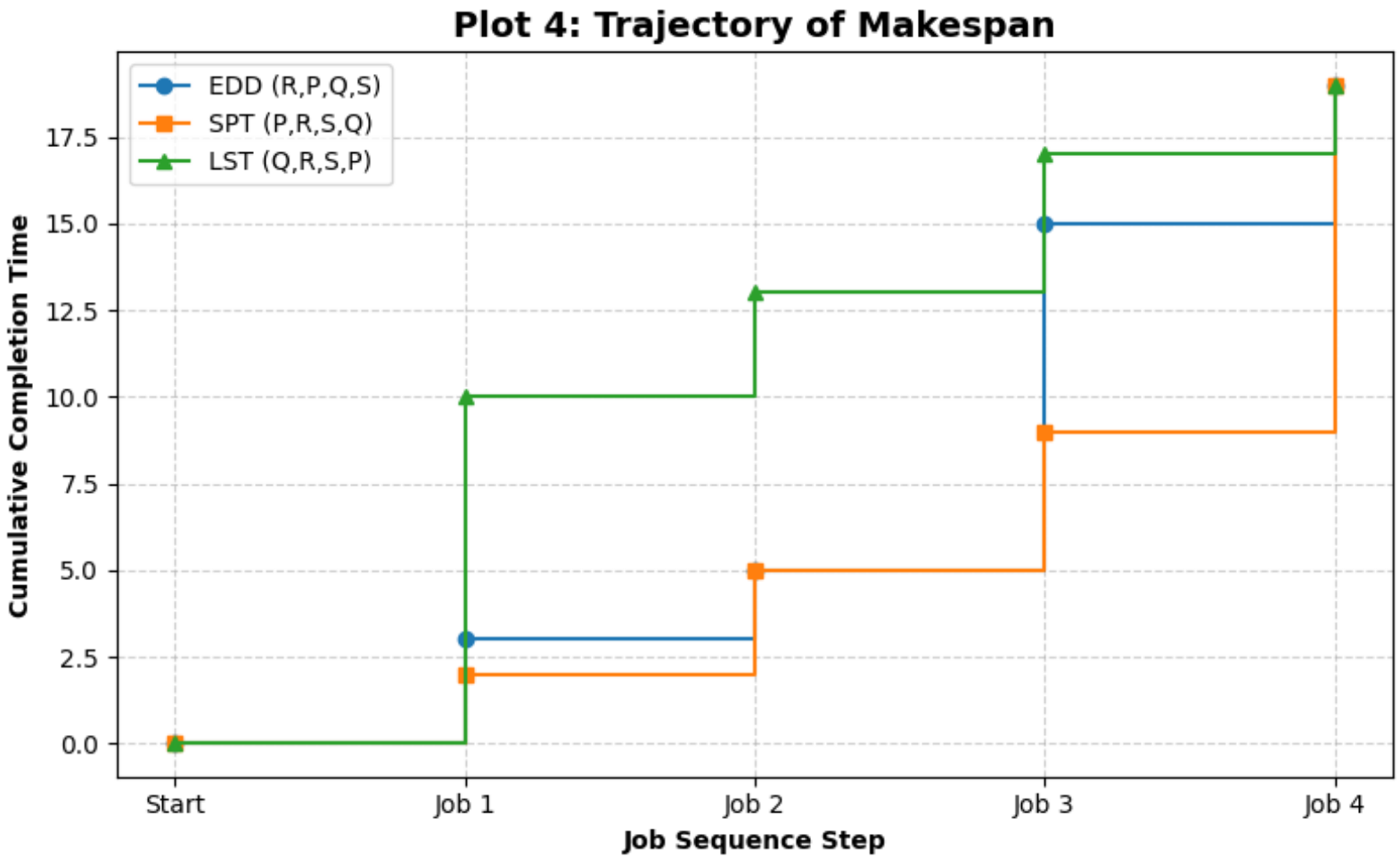
Python: Absolute Deviation

```
job_labels = ['P', 'Q', 'R', 'S']
x = np.arange(len(job_labels))
width = 0.25

_, abs_lat_edd, _, _ = analyze_sequence(seq_edd)
_, abs_lat_spt, _, _ = analyze_sequence(seq_spt)
_, abs_lat_lst, _, _ = analyze_sequence(seq_lst)

edd_vals = [abs_lat_edd[j] for j in job_labels]
spt_vals = [abs_lat_spt[j] for j in job_labels]
lst_vals = [abs_lat_lst[j] for j in job_labels]

fig2, ax2 = plt.subplots()
ax2.bar(x - width, edd_vals, width, label='EDD')
ax2.bar(x, spt_vals, width, label='SPT')
ax2.bar(x + width, lst_vals, width, label='LST')

plt.savefig("A.png")
```

Absolute Deviation Plot



Python: Completion Time Trajectory

```
_, _, _, cum_edd = analyze_sequence(seq_edd)
_, _, _, cum_spt = analyze_sequence(seq_spt)
_, _, _, cum_lst = analyze_sequence(seq_lst)

steps = [0, 1, 2, 3, 4]

fig4, ax4 = plt.subplots()
ax4.step(steps, cum_edd, label='EDD', where='post')
ax4.step(steps, cum_spt, label='SPT', where='post')
ax4.step(steps, cum_lst, label='LST', where='post')

ax4.legend()
plt.savefig("T.png")
```

Completion Time Trajectory